Breast Cancer Prediction Prototype

Project Guide

Breast Cancer Prediction Prototype

This is a learning project built with Python and Streamlit. It uses 30 cell nucleus measurements from the Wisconsin Diagnostic Breast Cancer (WDBC) dataset to predict whether a record is labeled **benign** or **malignant**.

> **Important:** This is a classroom demonstration, not a diagnostic tool. Its predictions are not medical advice or calibrated estimates of a person's cancer risk.

Run the app

If you have not downloaded the project yet, clone the repository and enter its folder:

```
git clone https://github.com/OtieteAyebanua/brestCancerdetection.git
cd brestCancerdetection
```

The project was run with Python 3.9.6. Install that version, then from the project folder create an environment and install the required packages.

macOS/Linux:

```
python3.9 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
streamlit run app.py --server.port 3001
```

Windows PowerShell:

```
py -3.9 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
streamlit run app.py --server.port 3001
```

Open http://localhost:3001. Stop the app with **Ctrl+C**. Once the environment is set up, you can also start the app from the project folder with `./run` on macOS/Linux.

Using the app

The app collects 30 measurements in three sections: **Mean**, **Standard error**, and **Worst**. Each section has 10 measurements. Enter all 30 values and choose **Get prediction**. The app displays its predicted class and the **probability of malignancy** estimated by the model. The probability is not calibrated as an individual risk estimate.

Choose **Load held-out demo case** to fill the form with one of six saved test examples. The app cycles through the examples. After you request a prediction, it displays the example's actual label beside the model's prediction. These examples are for demonstrating the app; they are not new clinical cases.

If a value is outside the range seen during training, the app warns that the prediction may be unreliable and still shows the model output. Use **Clear inputs** to reset the form. Input values stay in the active Streamlit session; this project does not include accounts, a database, or prediction history.

How the project works

1. `train_model.py` loads the WDBC dataset bundled with scikit-learn and splits its 569 records into training and test sets.
2. It tunes three classifiers—Logistic Regression, Support Vector Machine (SVM), and Random Forest—using cross-validation on the training set.
3. It selects one model using the cross-validation results, then evaluates all three models on the held-out test set.
4. It saves the selected model and supporting reports and data files.
5. `app.py` loads the saved model and uses it to make predictions. The app does not train a model when someone enters measurements.

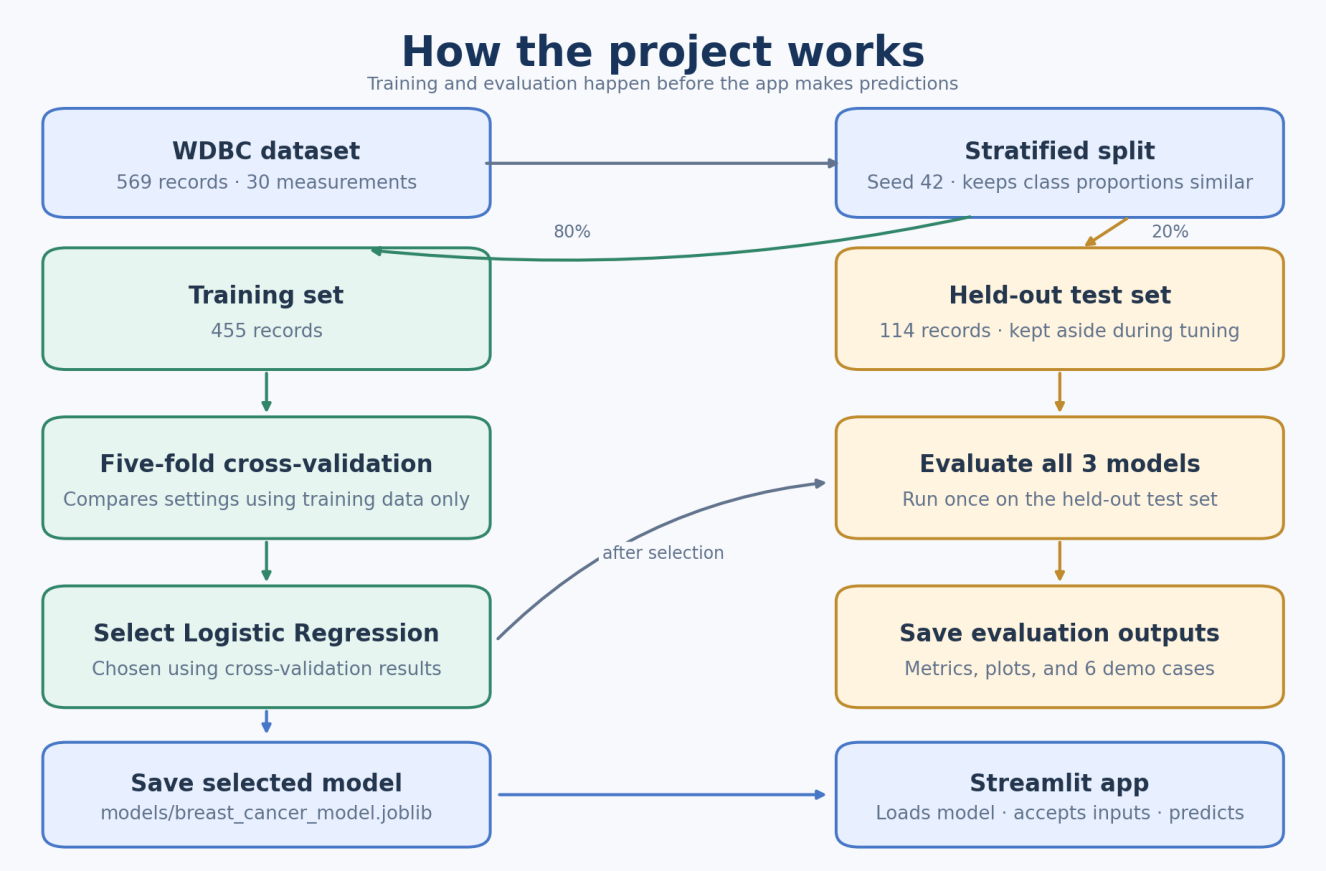


Figure 1: Diagram showing the WDBC dataset split into training and test sets, model comparison, test evaluation, and the saved model being used by the app.

Figure 1. Project workflow. The held-out test set is used for evaluation after cross-validation model selection is complete.

The split is stratified, meaning the training and test sets preserve approximately the same benign/malignant proportions as the full dataset. It uses random seed 42 so the split can be reproduced. The training file has 455 records; the test file has 114.

Model selection and results

The script uses five-fold stratified cross-validation on the training set. It selects among models whose mean cross-validation ROC-AUC is within 0.01 of the best, preferring higher malignant recall, then higher F1, then the simpler model according to a fixed order. It does not use the test set to tune or select a model.

The saved run selected **Logistic Regression**. These are the reported results from the one held-out split:

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9825	0.9762	0.9762	0.9762	0.9954
SVM	0.9825	0.9762	0.9762	0.9762	0.9977
Random Forest	0.9561	0.9512	0.9286	0.9398	0.9927

These scores describe one split of a small public dataset. They do not establish clinical performance or performance on other populations or hospitals.

Concepts to explain in your presentation

Features and labels: The 30 measurements are the model's features, or inputs. The dataset's `benign` or `malignant` value is the label the model learns to predict.

Training and test data: The training data is used to tune and fit models. The separate test data is held back until the models have been compared, so it can show how they perform on records that were not used for that selection. Here, the split is 455 training records and 114 test records.

Cross-validation: The training data is divided into five parts. The model is trained on four parts and checked on the fifth, rotating which part is held out. This gives a more stable basis for comparing model settings than one training/validation split. The test data is not part of cross-validation.

The three models: Logistic Regression finds a weighted boundary between the two classes. SVM (Support Vector Machine) finds a boundary that separates the classes, and can model curved boundaries. Random Forest combines the votes of many decision trees. These are three different ways to learn patterns from the same measurements.

How the model was chosen: The script first considers cross-validation ROC-AUC. It keeps models whose score is within 0.01 of the best, then favors higher malignant recall, higher F1, and finally a fixed model order. This is why the saved run chose Logistic Regression even though SVM had a slightly higher mean cross-validation ROC-AUC.

What the evaluation metrics mean: Accuracy is the share of all records classified correctly. Precision answers: of the records predicted malignant, how many were actually malignant? Recall answers: of the records that were actually malignant, how many did the model find? F1 combines precision and recall into one score. ROC-AUC summarizes how well the model ranks malignant records above benign ones across different decision thresholds.

Probability of malignancy: This is the model's estimated probability that an input belongs to the dataset's malignant class. It is not calibrated as a person's medical risk and must not be used as a diagnosis.

A short explanation you can say:

> "We used 30 measurements from the Wisconsin Diagnostic Breast Cancer dataset. We split the records into training and test data. We used five-fold cross-validation on the training data to compare Logistic Regression, SVM, and Random Forest, then evaluated the selected model on the held-out test data. The app loads the saved Logistic Regression model and displays its classification and estimated probability of malignancy. This is a learning prototype, not a clinical diagnostic tool."

Data files

The dataset is included with scikit-learn, so no separate download is needed. Each record has 30 numeric features and a target label (`malignant` or `benign`).

- `training_data.json` contains the 455 records used for cross-validation, model selection, and fitting.
- `testing_data.json` contains the 114 records held out for evaluation.

- demo_cases.json contains six examples sampled from the test set: three benign and three malignant. Each record includes the measurements, actual label, saved model prediction, and malignant probability.

The training and test JSON records include a `source_index`, a `features` object, and a `target` label. `demo_cases.json` is only a small example set; it is not the full test set. The model's reported test metrics were computed on all 114 test records.

Project files

File or folder	What it does
app.py	Runs the Streamlit interface, validates inputs, loads the saved model, and displays predictions. It also loads the held-out examples from `demo_cases.json`.
train_model.py	Splits the data, tunes and compares the three classifiers, evaluates them, and writes model, data, and report files.
requirements.txt	Lists the Python packages needed by the project.
.python-version	Selects Python 3.9.6 for tools that recognize this file.
run	Convenience launcher for macOS/Linux after Python and dependencies are set up.
project_workflow.png and project_workflow.svg	Workflow diagram embedded in this guide and its scalable vector version.
models/breast_cancer_model.joblib	The selected model loaded by the app.
models/model_evidence.json	Cross-validation, selection, and test evaluation details.
models/data_quality_report.json	Dataset checks and feature summaries.
models/roc_curves.png, models/confusion_matrices.png	Comparison plots created when you run `train_model.py`.
models/pip-freeze.txt	Package versions recorded for the model-generation environment.

To regenerate the model and reports, activate the project environment and run:

```
python train_model.py
```

This rebuilds the training and test JSON files, six demo cases, selected model, evidence report, data quality report, and comparison plots. The app needs the saved model file to make predictions and `demo_cases.json` to load the demo examples.

Limitations

- This project is for learning and demonstration, not clinical use.
- The dataset is a small public benchmark and cannot establish clinical validity.
- The probability shown by the app is a model output, not a calibrated patient risk estimate.
- The six demo cases come from the held-out test set and are examples for demonstrating predictions, not a new independent evaluation.
- The app has no patient accounts, database, or prediction history.